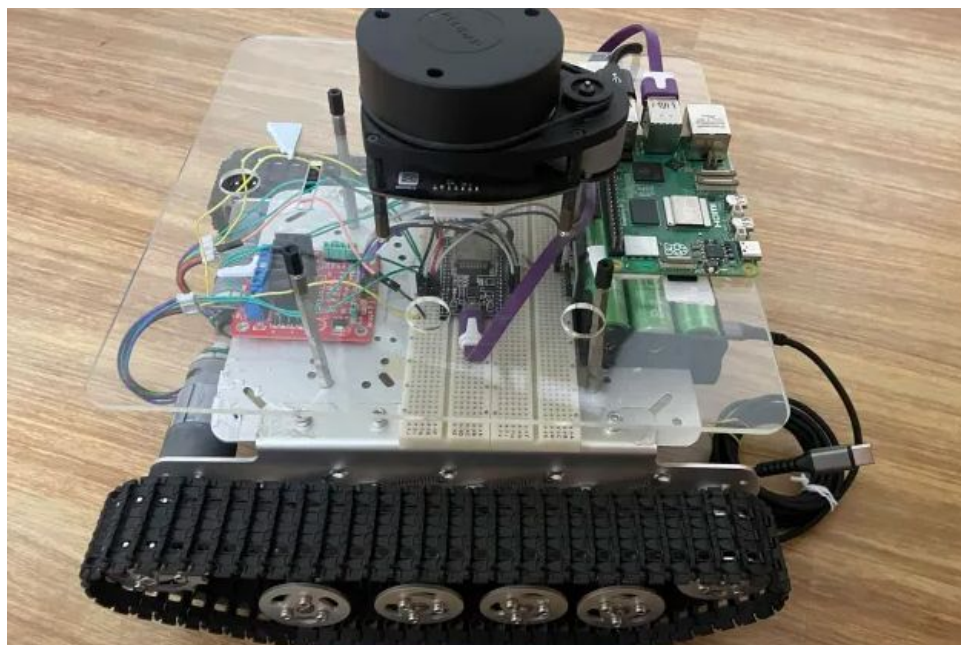


跨樓層自主移動機器人 AMR系統開發

基於ROS2設計的AMR系統，可用於軟體模擬自動導航與實體佈署。



國立成功大學工程科學系
學生：范舜傑 | 指導教授：侯廷偉
20260608

系統架構總覽

分層架構設計

AMR 系統採分層解耦設計，共分為四層：

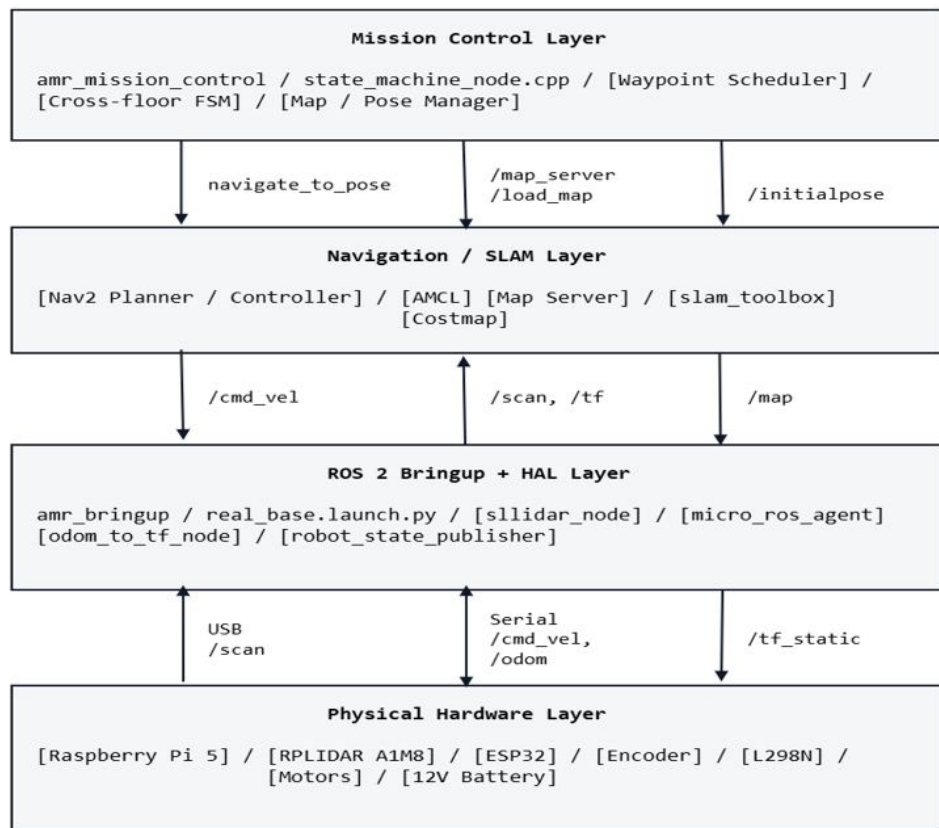
- **任務決策層**：決定任務流程 (Waypoint, FSM, 地圖切換)。
- **導航與 SLAM 層**：負責定位、路徑規劃與動態避障。
- **ROS 2 硬體整合層**：啟動 LiDAR、ESP32 與 TF 資料流。
- **實體硬體層**：感測器與馬達驅動實體組件。

核心資料流

LiDAR /scan → Nav2 (計算命令) → /cmd_vel → 馬達控制

分層設計讓軟體邏輯與硬體替換保持獨立，提升開發效率與維護性。

Cross-floor AMR System Architecture



Minimal Topic / TF Summary
/cmd_vel: Nav2 -> micro_ros_agent -> ESP32 -> Motors
/scan: RPLIDAR -> sllidar_node -> SLAM / AMCL / Costmap
/odom: ESP32 -> micro_ros_agent -> odom_to_tf_node
TF: map -> odom -> base_link -> laser_frame

ROS 2 通訊架構

ROS 2 通訊角色說明

本專題核心在於實現 ROS 2 於**軟體模擬**與**實體車**之間的通訊統一化。

軟體模擬端 (Simulation)

透過 ROS-Gazebo Bridge 將模擬資料 (/clock, /scan, /cmd_vel) 橋接至 ROS 2, 於 RViz 與 Nav2 中驗證導航演算法。

實體硬體端 (Hardware)

由 Raspberry Pi 5 啟動 `real_base`, 整合 LiDAR、ESP32 與座標轉換, 對外呈現完全一致的 Topic 介面。

重點: 統一的 Topic 與 TF 架構實現了 *Sim-to-Real* 的軟硬體解耦設計, 確保模擬環境的程式碼可直接無縫部署至實體車。

```
john@amr-pi:~$ cd ~/amr_ws
source /opt/ros/jazzy/setup.bash
source install/setup.bash
export ROS_DOMAIN_ID=0

ros2 daemon stop
ros2 daemon start
ros2 topic list
The daemon is not running
The daemon has been started
/cmd_vel
/joint_states
/odom
/parameter_events
/robot_description
/rosout
/scan
/tf
/tf_static
```

```
# 6. ROS-GZ Bridge (打通資料流：時鐘、雷達、速度、TF、關節狀態)
bridge = Node(
    package='ros_gz_bridge',
    executable='parameter_bridge',
    arguments=[
        '/clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock',
        '/scan@sensor_msgs/msg/LaserScan[gz.msgs.LaserScan',
        '/cmd_vel@geometry_msgs/msg/Twist[gz.msgs.Twist',
        '/tf@tf2_msgs/msg/TFMessage[gz.msgs.Pose_V',
        '/joint_states@sensor_msgs/msg/JointState[gz.msgs.Model' # <--- 新增此行
    ],
    output='screen'
)
```

模擬環境

軟體模擬環境配置

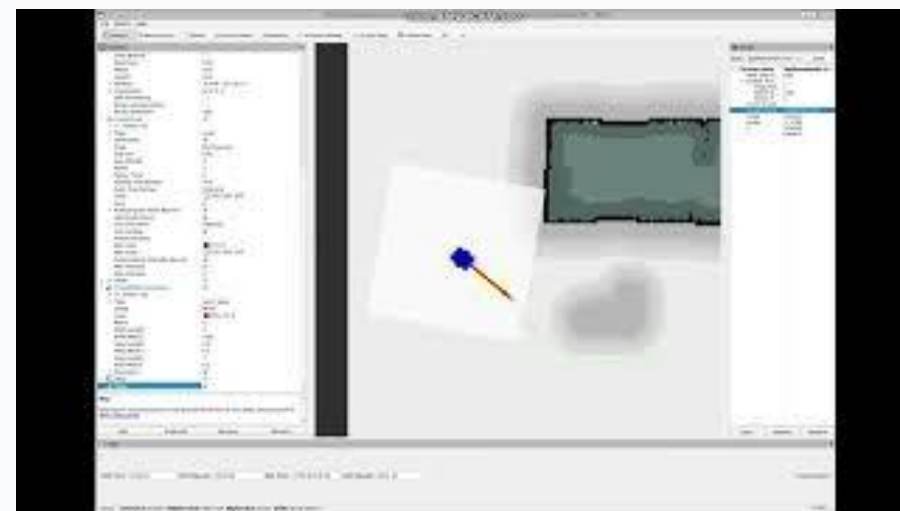
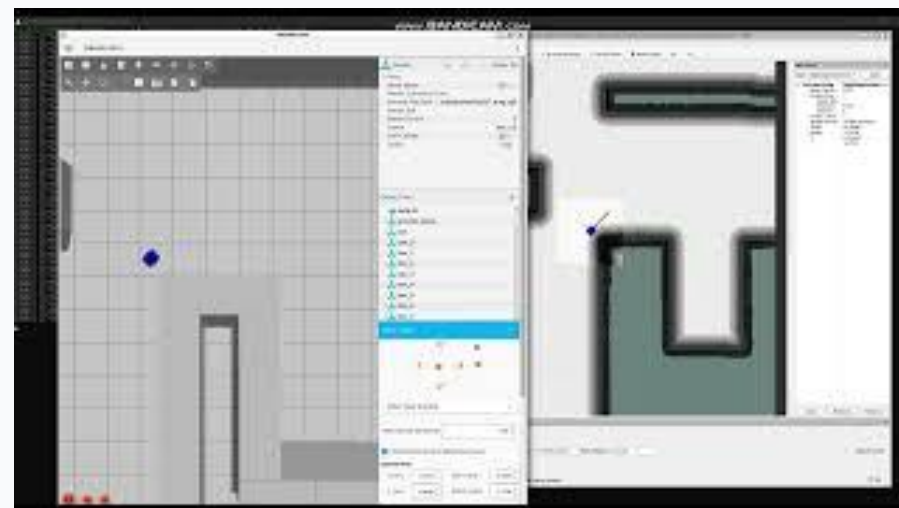
核心平台: WSL 為主地點, 搭載 Ubuntu 24.04 作業系統

開發架構: ROS 2 JAZZY 版本, 主要使用 C++ 進行開發撰寫

工具與監控: Gazebo 作為物理模擬器, Rviz2 負責資料監控

啟動方式: 使用 ROS 2 launch 進行節點管理與自動化執行

透過 WSL 與 ROS 2 的整合, 建立高效且可重複驗證的虛擬測試空間。



模擬環境

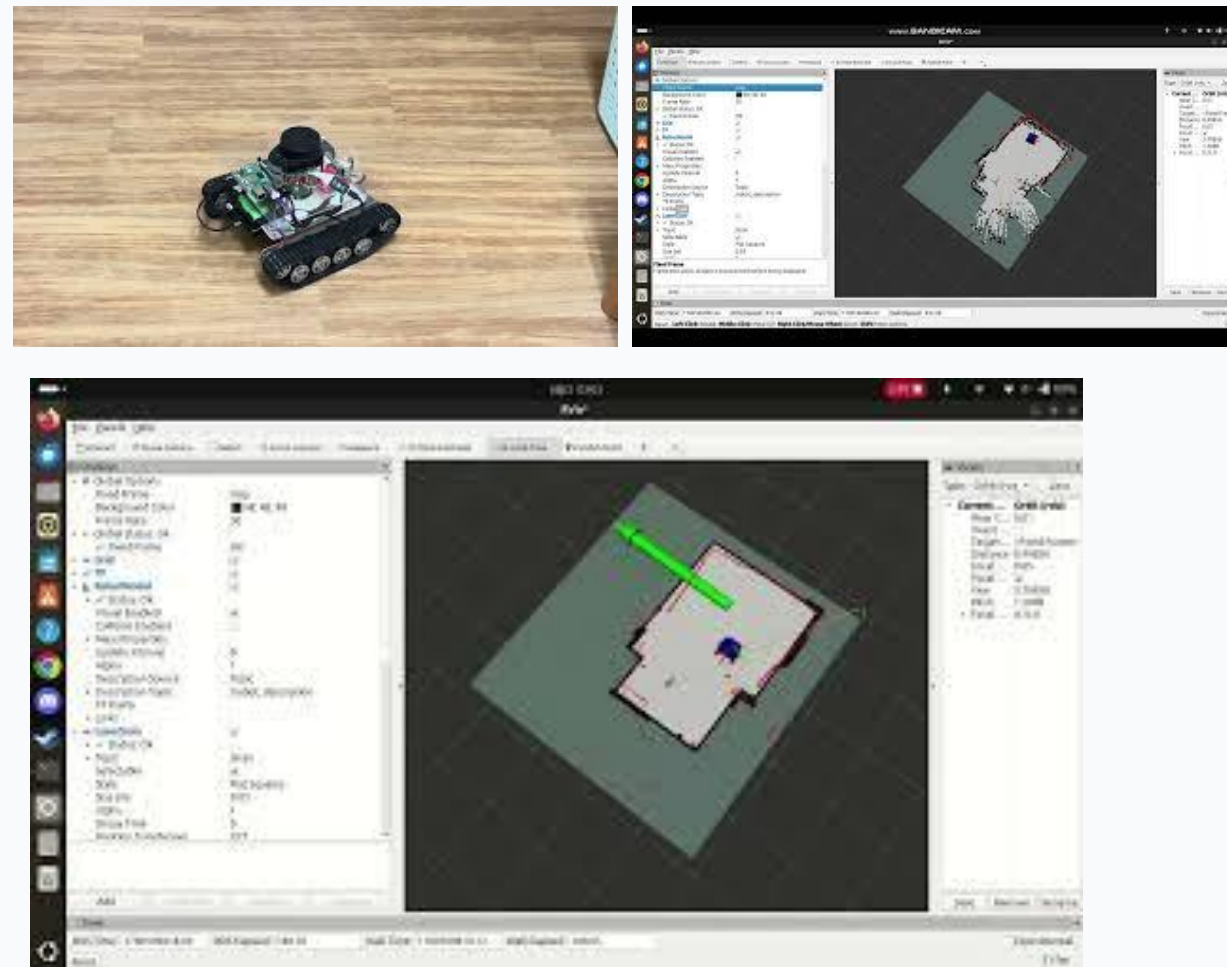
實體佈署配置

硬體核心組件：

- 樹莓派 (Raspberry Pi)
- ESP32 控制器
- MPU6050 慣性測量單元
- A1M8R5 激光雷達

執行與監控：

於樹莓派編譯與執行，透過**DDS** 通訊協議傳輸資料至筆記型電腦，並使用**Rviz2** 進行即時監控。



開發過程

1. 軟體模擬流程



2. 實體部署流程



此專題開發過程分為軟體模擬與硬體整合兩條主線。軟體端先建置ROS 2 Jazzy、Gazebo 與 RViz2 環境，接著撰寫 URDF / Xacro 建立 AMR 車體、輪組與 LiDAR 模型，並透過 ROS-Gazebo Bridge 串接 /scan、/cmd_vel、/tf 等通訊資料，完成模擬端的感測與控制驗證。

硬體端則從 Raspberry Pi 5 的 Ubuntu 24.04 與 ROS 2 環境建置開始，依序測試 RPLIDAR、ESP32、Encoder 與馬達控制，再透過 micro-ROS 將 ESP32 接入 ROS 2 系統。最後以 real_base.launch.py 整合 LiDAR、micro-ROS、TF 與 robot_state_publisher，完成實體車 /scan、/odom、/cmd_vel 與 /tf 的資料流驗證。

SLAM 建圖與定位流程 (軟體)

SLAM 建圖流程概述

本專題使用 RPLIDAR A1M8 作為主要環境感測器，透過 /scan topic 發布 2D 雷射掃描資料。SLAM 模組接收 /scan 與 TF 座標轉換後，逐步建立場域的 2D 佔用網格地圖。

完成建圖後，地圖可儲存為 map.yaml 與 map.pgm，後續由 Map Server 載入，並搭配 AMCL 進行定位，作為 Nav2 自主導航的基礎。

STEP 01

資料輸入與轉換

- /scan: LiDAR 雷射掃描資料
- /tf: 機器人座標轉換

STEP 02

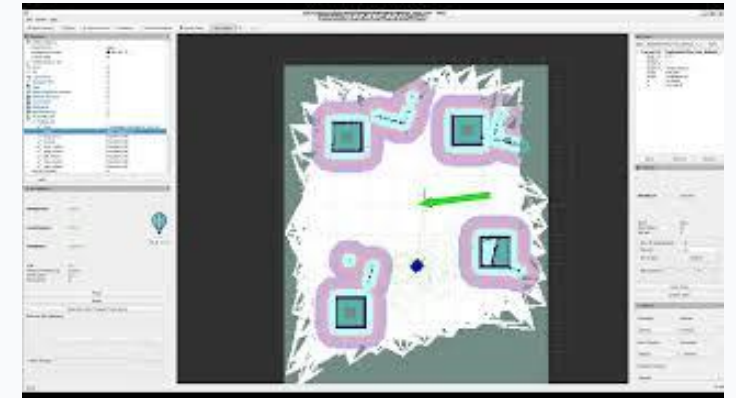
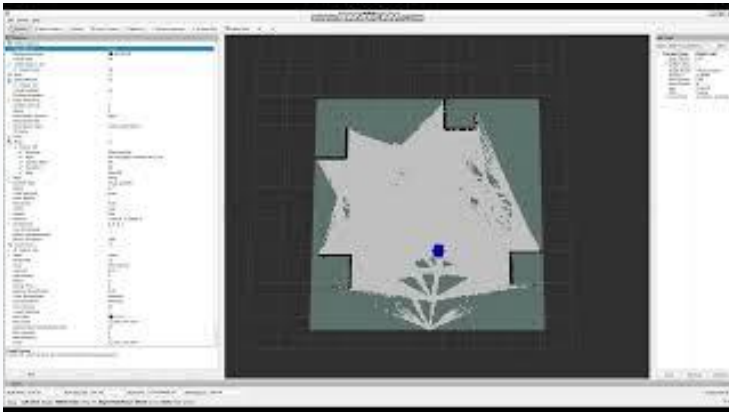
地圖建立與儲存

- slam_toolbox: 建立 2D 地圖
- map.yaml / pgm: 儲存地圖檔

STEP 03

定位與導航

- AMCL: 根據地圖與雷射定位
- Nav2: 進行自主導航



Nav2 自主導航流程 (實體)

Nav2 怎麼把目標點變成馬達控制概述

Nav2 負責將使用者指定的目標點轉換為實際底盤控制命令。系統會根據 Map Server 載入的地圖、AMCL 提供的定位結果, 以及 LiDAR 掃描建立的 Costmap 進行路徑規劃與避障, 最後輸出 `/cmd_vel` 給 ESP32 控制馬達, 使 AMR 能自主移動至指定位置。

STEP 01

目標點輸入

- 使用者或任務層發送 `NavigateToPose`
- 指定目標位置與方向

STEP 02

路徑規劃與避障

- Map Server 載入地圖
- AMCL 提供目前定位
- Costmap 建立障礙物資訊
- Planner 計算全域路徑
- Controller 產生控制命令

STEP 03

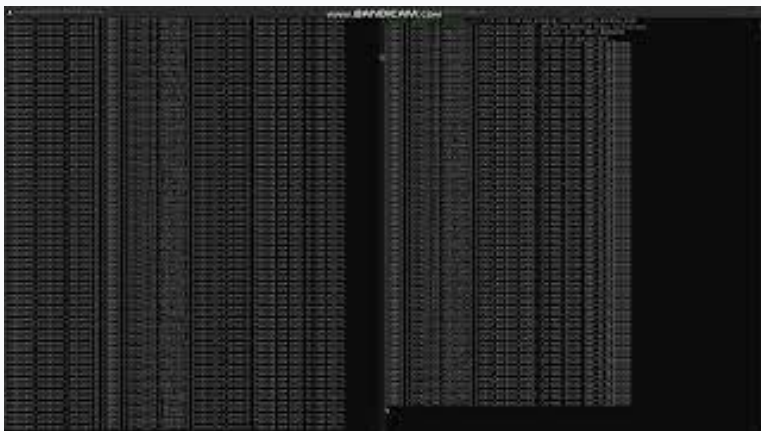
速度命令輸出

- Nav2 輸出 `/cmd_vel`
- ESP32 接收速度命令
- 轉換為馬達 PWM 控制
- 使 AMR 朝目標移動
- P.5 demo影片

跨樓層任務控制流程

跨樓層控制概念：

跨樓層任務以 **FSM 有限狀態機** 為核心，將導航程序拆解為多個階段，實現單層導航到跨樓層任務的延伸。



STEP 01

前往與進入

- 任務目標輸入
- NavigateToPose 前往電梯口
- 等待電梯 / 開門
- 進入電梯

STEP 02

樓層地圖切換

- LoadMap 切換新樓層地圖
- /initialpose 重新定位
- 完成新座標系初始化

STEP 03

執行新任務

- 離開電梯
- 發送新樓層目標點
- 接續發送下一個導航目標

```
enum class AmrState {  
    WAIT_ELEVATOR,  
    NAV_TO_ELEVATOR,  
    CHANGE_FLOOR_MAP,  
    IDLE  
};  
  
nav_client_ =  
    rclcpp_action::create_client<NavigateToPose>(  
        this, "navigate_to_pose");  
  
map_client_ =  
    this->create_client<LoadMap>(  
        "/map_server/load_map");
```

NavigateToPose: 發送導航目標
LoadMap: 切換樓層地圖
Timer: 每 2 秒更新任務狀態
Result Callback: 導航成功後進入地圖切換狀態

軟體開發成果

模擬開發

環境與模型建置

- 已完成 ROS 2 Jazzy、Gazebo 與 RViz2 開發環境建置。
- 透過 URDF / Xacro 建立 AMR 車體、輪組與 LiDAR 模型。

通訊串接測試

- 使用 ROS-Gazebo Bridge 串接 /clock, /scan, /cmd_vel, /tf 資料。
- 實現模擬端之感測、控制與導航測試功能。

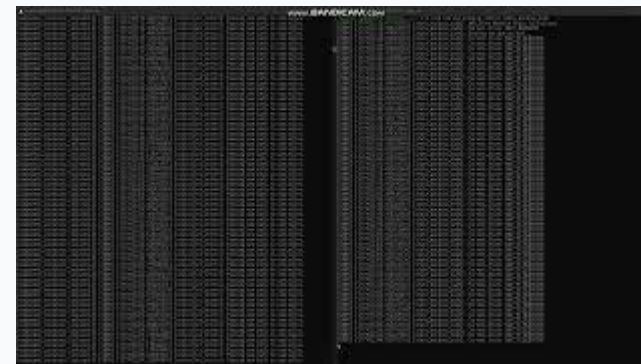
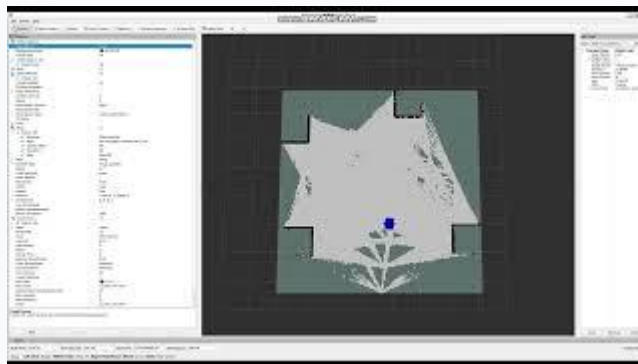
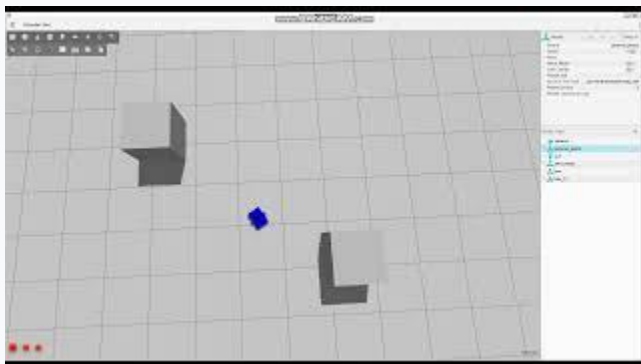
導航驗證

導航功能測試

- 完成 SLAM 建圖流程與 Map Server 地圖載入。
- 成功執行 AMCL 定位與 Nav2 路徑規劃測試與切換地圖。

驗證平台價值

- 確認模擬環境可作為實體部署前的高效驗證平台，降低實體部署風險。



實體部署成果

硬體與通訊架構

主要運算平台: Raspberry Pi 5

- **感測整合:** 整合 RPLIDAR A1M8、ESP32、Encoder 與 L298N 馬達驅動板。
- **動力來源:** 採用 12V 動力電源。
- **ROS 2 角色:** 負責啟動 ROS 2 節點、管理感測資料與執行主要運算。
- **micro-ROS 通訊:** 透過 ESP32 接收 /cmd_vel 命令並回傳 /odom 資料。

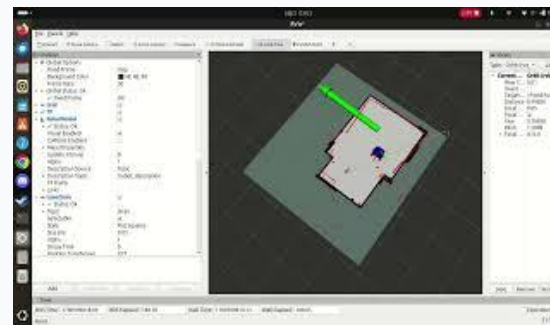
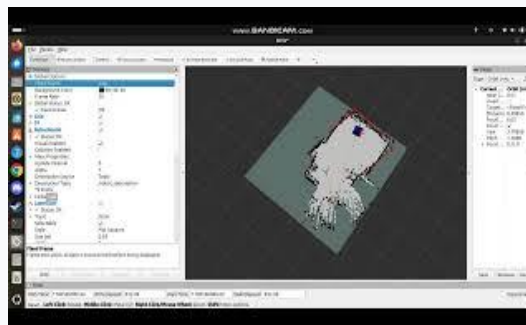
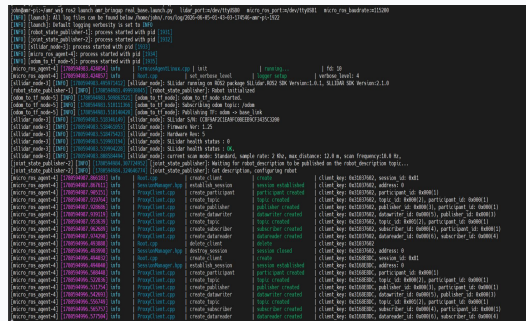
啟動整合與功能

Launch 文件: real_base.launch.py

- **啟動節點:** 整合 LiDAR、micro-ROS Agent、robot_state_publisher 與 odom_to_tf_node。
- **資料發布:** 使實體車能發布 /scan、/odom、/tf 與 /tf_static。
- **通訊介面:** 建立 Nav2 與底盤控制之間的基本通訊介面, 確保導航順暢。

問題與缺陷

由於車體控制系統沒有得到妥善的設計與補救措施，導致物理上移動的誤差值依然很大且不可控，造成運行 Nav2時會不斷左右搖擺調整自身角度。



詳細記錄與參考資料

參考資料與詳細紀錄

一、專題原始碼與開發紀錄

- [\[1\] AMR Software Workspace GitHub Repository](#)
- [\[2\] 專題開發紀錄\(一\)系統架構、模擬環境與 ROS 2 通訊](#)
- [\[3\] 專題開發紀錄\(二\)Raspberry Pi 5、LiDAR、ESP32 與實體部署](#)
- [\[4\] 專題開發紀錄\(三\)實體車資料流驗證、URDF 修正與馬達供電除錯](#)

二、ROS 2、Nav2 與導航系統文件

- [\[5\] ROS 2 Jazzy Documentation](#)
- [\[6\] Nav2 Documentation](#)
- [\[7\] Nav2 Parameters 整理筆記](#)

三、硬體控制與感測器參考

- [\[8\] PWM 控制原理與嵌入式系統應用](#)
- [\[9\] 磁性編碼器原理與應用資料](#)
- [\[10\] Graham C. Goodwin, Stefan F. Graebe, Mario E. Salgado, 控制系統設計, Prentice Hall PTR.](#)